Diseño y desarrollo de servicios web
Proyecto JXSTech

Aprendiz: Sergio Murillo
Evidencia: GA7-220501096-AA5-EV03
Actividad: Diseño y desarrollo de servicios web - proyecto
Fecha: 3 de noviembre de 2026
Versión del software: 2.0.0

1. Propósito y alcance

JXSTech es una tienda que vende relojes y audífonos inteligentes. Se desarrollaron servicios web para administrar operadores, clientes, catálogo de productos, inventario y ventas. La API recibe solicitudes HTTP y devuelve respuestas JSON.

La solución amplía el registro y el inicio de sesión de EV01 con las funciones comerciales de la tienda. Los campos de clientes corresponden al módulo trabajado en evidencias anteriores. Los datos se conservan en MongoDB.

El proyecto incluye código comentado, 24 operaciones documentadas, una especificación OpenAPI, una colección de Postman y pruebas. La página HTML demuestra el acceso de operadores; la operación comercial se comprueba mediante Postman.

2. Diseño general

Flujo: Postman o navegador > API Express > validación y sesión > módulo comercial > MongoDB > respuesta JSON.

Elemento	Responsabilidad
Node.js y Express	Ejecutar el servidor y organizar las rutas HTTP.
MongoDB Atlas	Guardar operadores, sesiones, clientes, productos, movimientos y ventas.
bcryptjs	Generar y verificar hashes de contraseñas.
Sesiones Bearer	Identificar al operador en los servicios protegidos.
Git	Conservar el historial de cambios; la rama de EV03 es aa5-ev03-jxstech.

3. Reglas de negocio

- Los productos pertenecen a las categorías reloj o audifonos y tienen SKU único. El precio se expresa en pesos colombianos enteros y debe estar entre 1 y 1.000.000.000 COP.

- El stock inicial es cero. Las unidades ingresan y salen mediante movimientos con motivo, operador, fecha y saldos. El rango de existencias es de 0 a 1.000.000 por producto.

- Cada venta requiere un cliente activo y de 1 a 50 productos distintos, activos y con existencias suficientes. Las cantidades son enteros positivos.

- El servidor obtiene los precios del catálogo y calcula subtotales y total. Una venta puede sumar como máximo 1.000.000.000.000 COP. El cliente HTTP no puede imponer el precio de los items.

- La referencia de venta es única. Repetirla devuelve 409 y evita otro descuento de inventario.

- La venta, los descuentos y los movimientos se guardan en una sola transacción. Si una línea falla, se revierte la operación completa.

- La anulación conserva el registro y devuelve las existencias una sola vez. Si se excede el máximo de stock, la anulación se rechaza y no se aplican cambios parciales.

- Las bajas son lógicas. Los clientes y productos pueden desactivarse y reactivarse. Un producto debe tener stock cero para desactivarse. Los registros inactivos permanecen en los listados.

- Los datos del cliente y los precios del momento se copian en cada venta para conservar su historial. El documento y correo del cliente también son únicos.

- Todos los operadores registrados tienen los mismos permisos en esta versión académica local. Se registra el medio de pago; no se integran pasarelas, envíos, impuestos ni facturación electrónica.

4. Modelo de datos

Colección	Contenido y relaciones
usuarios	Nombre de usuario único, hash de contraseña y fecha de registro.
sesiones	Hash del token, usuarioId, usuario y fecha de expiración.
clientes	Nombre, apellido, documento, correo, teléfono, dirección, estado y fechas.
productos	SKU, nombre, categoría, marca, descripción, precioCOP, stock, estado y fechas.
movimientos	productoId, variación, saldos, motivo, operadorId, fecha y ventaId cuando corresponde.
ventas	Referencia única, clienteId, datos históricos del cliente, items, totalCOP, estado y operadorId.

5. Convenciones y acceso

Dirección base: http://localhost:3001/api. Cuerpos de solicitudes y respuestas: application/json. El servidor escucha en 127.0.0.1.

El login entrega un token opaco de 64 caracteres válido por 8 horas. Las rutas comerciales reciben el encabezado Authorization: Bearer <token>. El servidor guarda el hash SHA-256 del token y comprueba su expiración en cada solicitud. Logout invalida la sesión.

Los identificadores son cadenas hexadecimales de 24 caracteres. Las fechas se devuelven en UTC e ISO 8601. Los cuerpos JSON tienen un límite de 32 KiB. Se rechazan campos de entrada no definidos.

Los listados aceptan pagina (1 a 100.000, predeterminado 1) y limite (1 a 100, predeterminado 20). Devuelven datos, total, pagina y limite. Los importes se expresan en COP enteros.

HTTP	Significado
200	Operación o consulta correcta.
201	Registro creado correctamente.
400	Datos, parámetros, identificadores o JSON inválidos.
401	Credenciales incorrectas o sesión ausente, inválida o expirada.
404	Ruta o registro no encontrado; algunas operaciones también rechazan registros inactivos.
409	Duplicado, existencias insuficientes o conflicto de estado.
413	El cuerpo supera el tamaño permitido.
429	Más de 30 solicitudes combinadas de registro/login por IP en 15 minutos.
500	Error interno sin revelar datos privados.

Formato general de error:

```
{
  "mensaje": "Descripción del error."
}
```

6. Catálogo de servicios

N.º	Método	Ruta y finalidad
1	GET	/api/salud Consultar disponibilidad
2	POST	/api/usuarios/registro Registrar operador
3	POST	/api/usuarios/login Iniciar sesión
4	GET	/api/usuarios/sesion Consultar sesión
5	POST	/api/usuarios/logout Cerrar sesión
6	POST	/api/clientes Registrar cliente
7	GET	/api/clientes Listar clientes
8	GET	/api/clientes/{id} Consultar cliente
9	PUT	/api/clientes/{id} Actualizar cliente
10	DELETE	/api/clientes/{id} Desactivar cliente
11	PATCH	/api/clientes/{id}/activar Activar cliente
12	POST	/api/productos Registrar producto
13	GET	/api/productos Listar productos
14	GET	/api/productos/{id} Consultar producto
15	PUT	/api/productos/{id} Actualizar producto
16	DELETE	/api/productos/{id} Desactivar producto
17	PATCH	/api/productos/{id}/activar Activar producto
18	GET	/api/inventario Consultar existencias
19	GET	/api/inventario/movimientos Consultar historial de inventario
20	POST	/api/inventario/movimientos Registrar entrada o salida
21	GET	/api/ventas Listar ventas
22	GET	/api/ventas/{id} Consultar venta
23	POST	/api/ventas Registrar venta
24	PATCH	/api/ventas/{id}/anular Anular venta

7. Documentación individual de los servicios

Los ejemplos utilizan información ficticia. Los identificadores de ejemplo deben sustituirse por los devueltos al crear los registros. Los modelos completos de respuesta están definidos en docs/openapi.json.

7.1. Consultar disponibilidad

GET /api/salud

Comprueba que la API puede comunicarse con MongoDB.

Autenticación: no requiere token.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: Salud. Objeto con estado, proyecto y version.

Errores previstos: 500. Su significado se explica en la tabla de códigos HTTP.

7.2. Registrar operador

POST /api/usuarios/registro

Crea una cuenta de operador y guarda un hash bcrypt de la contraseña. Todos los operadores registrados tienen acceso a los módulos comerciales de esta versión académica local.

Autenticación: no requiere token.

Campo	Tipo y validación	Obligatorio
usuario	string; 3 a 30 caracteres	Sí
password	string; 8 a 72 caracteres; Máximo 72 bytes UTF-8.	Sí

Ejemplo de solicitud

```
{
  "usuario": "operador_demo",
  "password": "PruebaSena2026"
}
```

Respuesta correcta: HTTP 201. Modelo: Mensaje. Objeto con mensaje.

Errores previstos: 400, 409, 413, 429, 500. Su significado se explica en la tabla de códigos HTTP.

7.3. Iniciar sesión

POST /api/usuarios/login

Comprueba las credenciales y entrega un token de 64 caracteres válido durante 8 horas. MongoDB conserva únicamente el hash SHA-256 del token.

Autenticación: no requiere token.

Campo	Tipo y validación	Obligatorio
usuario	string; 3 a 30 caracteres; Se recortan espacios externos y se convierte a minúsculas.	Sí
password	string; 1 a 72 caracteres; Registro: mínimo 8 caracteres. Máximo 72 bytes UTF-8; en login basta con que no esté vacía.	Sí

Ejemplo de solicitud

```
{
  "usuario": "operador_demo",
  "password": "PruebaSena2026"
}
```

Respuesta correcta: HTTP 200. Modelo: Login. Objeto con mensaje, token, tipo y expira.

Errores previstos: 400, 401, 413, 429, 500. Su significado se explica en la tabla de códigos HTTP.

7.4. Consultar sesión

GET /api/usuarios/sesion

Devuelve el identificador y el nombre del operador autenticado.

Autenticación: token Bearer de operador.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: Sesion. Objeto datos con _id y usuario del operador.

Errores previstos: 401, 500. Su significado se explica en la tabla de códigos HTTP.

7.5. Cerrar sesión

POST /api/usuarios/logout

Elimina la sesión del token enviado. Los siguientes usos de ese token serán rechazados.

Autenticación: token Bearer de operador.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: Mensaje. Objeto con mensaje.

Errores previstos: 401, 500. Su significado se explica en la tabla de códigos HTTP.

7.6. Registrar cliente

POST /api/clientes

Guarda los mismos campos comerciales del módulo de clientes desarrollado en evidencias anteriores.

Autenticación: token Bearer de operador.

Campo	Tipo y validación	Obligatorio
nombre	string; 2 a 60 caracteres	Sí
apellido	string; 2 a 60 caracteres	Sí
documento	string; 5 a 20 caracteres	Sí
correo	string; 5 a 120 caracteres	Sí
telefono	string; 7 a 20 caracteres	Sí
direccion	string; 5 a 150 caracteres	Sí

Ejemplo de solicitud

```
{
  "nombre": "Laura",
  "apellido": "Prueba",
  "documento": "1000123456",
  "correo": "laura@example.com",
  "telefono": "3001234567",
  "direccion": "Calle 10 numero 20"
}
```

Respuesta correcta: HTTP 201. Modelo: DatosCliente. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 409, 413, 500. Su significado se explica en la tabla de códigos HTTP.

7.7. Listar clientes

GET /api/clientes

Devuelve registros activos e inactivos en una lista paginada, ordenada por identificador descendente.

Autenticación: token Bearer de operador.

Consulta opcional: pagina y limite.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: ListaCliente. Objeto con datos (arreglo de registros), total, pagina y limite.

Errores previstos: 400, 401, 500. Su significado se explica en la tabla de códigos HTTP.

7.8. Consultar cliente

GET /api/clientes/{id}

Busca un registro por identificador, incluyendo registros desactivados.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: DatosCliente. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 500. Su significado se explica en la tabla de códigos HTTP.

7.9. Actualizar cliente

PUT /api/clientes/{id}

Actualiza todos los campos del cliente activo; el documento y el correo deben conservar su unicidad.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Campo	Tipo y validación	Obligatorio
nombre	string; 2 a 60 caracteres	Sí
apellido	string; 2 a 60 caracteres	Sí
documento	string; 5 a 20 caracteres	Sí
correo	string; 5 a 120 caracteres	Sí
telefono	string; 7 a 20 caracteres	Sí
direccion	string; 5 a 150 caracteres	Sí

Ejemplo de solicitud

```
{
  "nombre": "Laura",
  "apellido": "Prueba",
  "documento": "1000123456",
  "correo": "laura@example.com",
  "telefono": "3001234567",
  "direccion": "Calle 10 numero 20"
}
```

Respuesta correcta: HTTP 200. Modelo: DatosCliente. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 409, 413, 500. Su significado se explica en la tabla de códigos HTTP.

7.10. Desactivar cliente

DELETE /api/clientes/{id}

Baja lógica del cliente. Conserva el registro y bloquea ventas nuevas para ese cliente.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: DatosCliente. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 500. Su significado se explica en la tabla de códigos HTTP.

7.11. Activar cliente

PATCH /api/clientes/{id}/activar

Reactiva el registro conservando su identificador e historial. Si ya está activo, responde 200.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: DatosCliente. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 500. Su significado se explica en la tabla de códigos HTTP.

7.12. Registrar producto

POST /api/productos

Crea un reloj o audífonos inteligentes. El stock inicial es cero; se modifica mediante inventario y ventas.

Autenticación: token Bearer de operador.

Campo	Tipo y validación	Obligatorio
sku	string; 3 a 40 caracteres; Se convierte a mayúsculas; es único.	Sí
nombre	string; 3 a 100 caracteres	Sí
categoria	reloj / audifonos	Sí
marca	string; 2 a 60 caracteres	Sí
descripcion	string; 0 a 500 caracteres	No
precioCOP	integer; entero de 1 a 1000000000	Sí

Ejemplo de solicitud

```
{
  "sku": "REL-001",
  "nombre": "Reloj inteligente",
  "categoria": "reloj",
  "marca": "Demo",
  "descripcion": "Producto de demostración",
  "precioCOP": 150000
}
```

Respuesta correcta: HTTP 201. Modelo: DatosProducto. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 409, 413, 500. Su significado se explica en la tabla de códigos HTTP.

7.13. Listar productos

GET /api/productos

Devuelve registros activos e inactivos en una lista paginada, ordenada por identificador descendente.

Autenticación: token Bearer de operador.

Consulta opcional: pagina y limite.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: ListaProducto. Objeto con datos (arreglo de registros), total, pagina y limite.

Errores previstos: 400, 401, 500. Su significado se explica en la tabla de códigos HTTP.

7.14. Consultar producto

GET /api/productos/{id}

Busca un registro por identificador, incluyendo registros desactivados.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: DatosProducto. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 500. Su significado se explica en la tabla de códigos HTTP.

7.15. Actualizar producto

PUT /api/productos/{id}

Actualiza los datos del catálogo de un producto activo. No recibe stock. Conserva los precios históricos de las ventas.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Campo	Tipo y validación	Obligatorio
sku	string; 3 a 40 caracteres; Se convierte a mayúsculas; es único.	Sí
nombre	string; 3 a 100 caracteres	Sí
categoria	reloj / audifonos	Sí
marca	string; 2 a 60 caracteres	Sí
descripcion	string; 0 a 500 caracteres	No
precioCOP	integer; entero de 1 a 1000000000	Sí

Ejemplo de solicitud

```
{
  "sku": "REL-001",
  "nombre": "Reloj inteligente",
  "categoria": "reloj",
  "marca": "Demo",
  "descripcion": "Producto de demostración",
  "precioCOP": 150000
}
```

Respuesta correcta: HTTP 200. Modelo: DatosProducto. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 409, 413, 500. Su significado se explica en la tabla de códigos HTTP.

7.16. Desactivar producto

DELETE /api/productos/{id}

Baja lógica del producto. Requiere stock igual a cero; conserva el historial.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: DatosProducto. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 409, 500. Su significado se explica en la tabla de códigos HTTP.

7.17. Activar producto

PATCH /api/productos/{id}/activar

Reactiva el registro conservando su identificador e historial. Si ya está activo, responde 200.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: DatosProducto. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 500. Su significado se explica en la tabla de códigos HTTP.

7.18. Consultar existencias

GET /api/inventario

Lista los productos con sus existencias actuales, incluidos los inactivos, con paginación.

Autenticación: token Bearer de operador.

Consulta opcional: pagina y limite.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: ListaProducto. Objeto con datos (arreglo de registros), total, pagina y limite.

Errores previstos: 400, 401, 500. Su significado se explica en la tabla de códigos HTTP.

7.19. Consultar historial de inventario

GET /api/inventario/movimientos

Lista entradas, salidas, ventas y anulaciones. Acepta productoId como filtro opcional y ordena por fecha descendente.

Autenticación: token Bearer de operador.

Consulta opcional: pagina y limite.

Filtro opcional adicional: productoId.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: ListaMovimiento. Objeto con datos (arreglo de registros), total, pagina y limite.

Errores previstos: 400, 401, 500. Su significado se explica en la tabla de códigos HTTP.

7.20. Registrar entrada o salida

POST /api/inventario/movimientos

Registra un ajuste justificado y modifica el saldo en una transacción. Impide existencias negativas o superiores a 1.000.000. Los tipos venta y anulacion son exclusivos del sistema.

Autenticación: token Bearer de operador.

Campo	Tipo y validación	Obligatorio
productoId	string; 24 caracteres hexadecimales	Sí
tipo	entrada / salida	Sí
cantidad	integer; entero de 1 a 1000000	Sí
motivo	string; 5 a 200 caracteres	Sí

Ejemplo de solicitud

```
{
  "productoId": "507f1f77bcf86cd799439012",
  "tipo": "entrada",
  "cantidad": 10,
  "motivo": "Ingreso de mercancía para la tienda"
}
```

Respuesta correcta: HTTP 201. Modelo: DatosMovimiento. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 409, 413, 500. Su significado se explica en la tabla de códigos HTTP.

7.21. Listar ventas

GET /api/ventas

Lista ventas registradas y anuladas mediante paginación.

Autenticación: token Bearer de operador.

Consulta opcional: pagina y limite.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: ListaVenta. Objeto con datos (arreglo de registros), total, pagina y limite.

Errores previstos: 400, 401, 500. Su significado se explica en la tabla de códigos HTTP.

7.22. Consultar venta

GET /api/ventas/{id}

Devuelve el detalle, los precios históricos, total, estado, cliente y operador de una venta.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Cuerpo de entrada: no requiere.

Respuesta correcta: HTTP 200. Modelo: DatosVenta. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 500. Su significado se explica en la tabla de códigos HTTP.

7.23. Registrar venta

POST /api/ventas

Valida el cliente activo, los productos activos y el stock. Calcula el total con los precios del catálogo. Guarda venta, detalles y movimientos en una sola transacción. Si falla una línea, revierte todos los cambios. Cada referencia se puede registrar una sola vez.

Autenticación: token Bearer de operador.

Campo	Tipo y validación	Obligatorio
referencia	string; 3 a 64 caracteres; Identificador comercial único. Se normaliza a mayúsculas. Repetirlo devuelve 409 sin otro descuento.	Sí
clienteId	string; 24 caracteres hexadecimales	Sí
medioPago	efectivo / transferencia / tarjeta	Sí
items	array; 1 a 50 productos distintos con productoId y cantidad entera de 1 a 1.000.000	Sí

Ejemplo de solicitud

```
{
  "referencia": "VENTA-001",
  "clienteId": "507f1f77bcf86cd799439011",
  "medioPago": "efectivo",
  "items": [
    {
      "productoId": "507f1f77bcf86cd799439012",
      "cantidad": 2
    }
  ]
}
```

Respuesta correcta: HTTP 201. Modelo: DatosVenta. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 409, 413, 500. Su significado se explica en la tabla de códigos HTTP.

7.24. Anular venta

PATCH /api/ventas/{id}/anular

Conserva la venta, registra el motivo y restituye todas las existencias en una transacción. La restitución solo puede suceder una vez. También admite productos desactivados; pueden reactivarse después.

Autenticación: token Bearer de operador.

Ruta: id, cadena hexadecimal de 24 caracteres.

Campo	Tipo y validación	Obligatorio
motivo	string; 5 a 200 caracteres	Sí

Ejemplo de solicitud

```
{
  "motivo": "El cliente cancela su compra"
}
```

Respuesta correcta: HTTP 200. Modelo: DatosVenta. Objeto datos con el registro; las operaciones de escritura también devuelven mensaje.

Errores previstos: 400, 401, 404, 409, 413, 500. Su significado se explica en la tabla de códigos HTTP.

8. Instalación y ejecución

1. Extraer el proyecto y abrirlo en Visual Studio Code. Se necesita Node.js 20.19 o posterior y MongoDB Atlas o un replica set.

2. Dentro de backend, crear .env a partir de .env.example. Completar la URI privada de Atlas y utilizar DB_NAME=jxstech_ev03 y PORT=3001.

3. Desde backend ejecutar npm.cmd ci y después npm.cmd start. La dirección de comprobación es http://localhost:3001/api/salud.

4. Importar postman/JXSTech_EV03.postman_collection.json y ejecutar las 36 solicitudes en orden, en modo Functional y con una iteración.

5. La colección genera datos ficticios nuevos, conserva identificadores y token en variables, y prueba desde el registro hasta el cierre de sesión. Los datos de esa demostración permanecen en la base configurada.

6. Para la suite automatizada, ejecutar npm.cmd test desde backend. Esta suite crea y elimina únicamente su propia base temporal; necesita permisos de Atlas para hacerlo.

9. Verificación del funcionamiento

Durante la preparación se aprobaron 38 casos de integración contra MongoDB real. La colección también se verificó con Newman: 36 solicitudes y 82 comprobaciones aprobadas, sin fallos. Los informes detallados están incluidos en docs/resultados-pruebas.json y docs/resultados-postman.json.

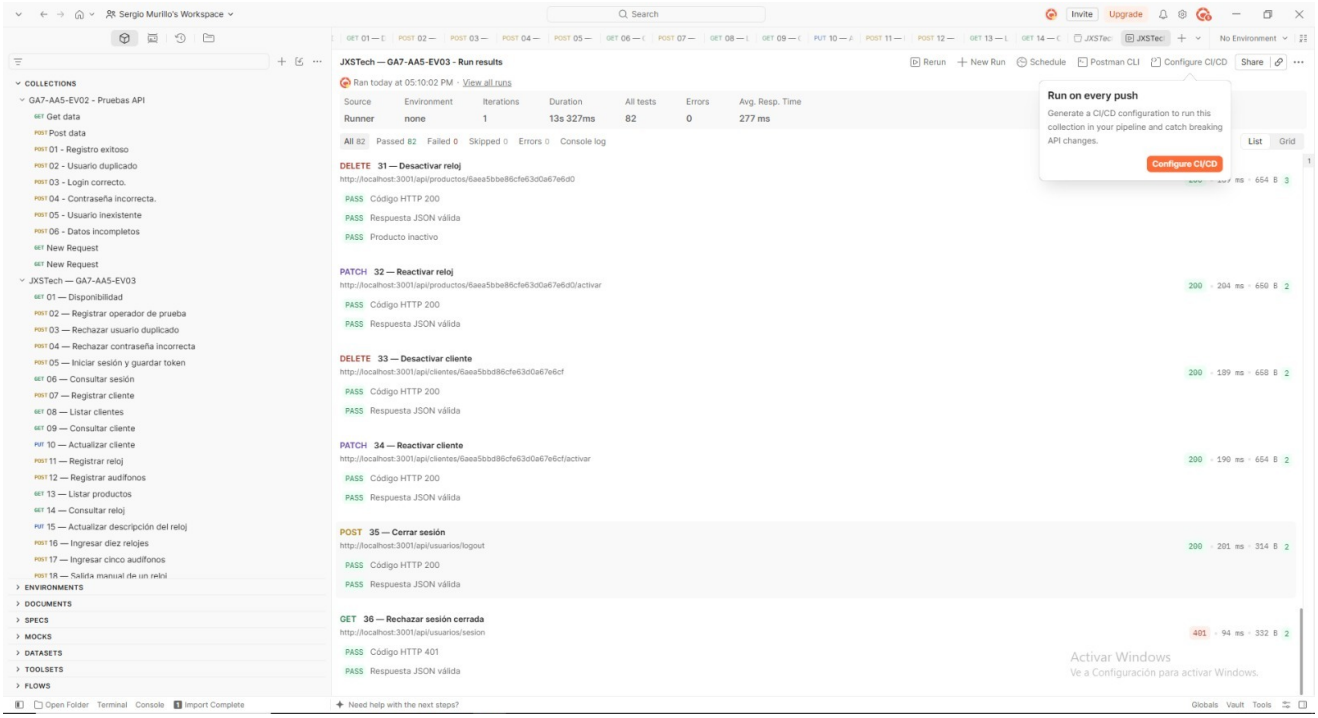
Además, el aprendiz ejecutó la colección completa en Postman el 3 de noviembre de 2026. La captura del resumen muestra Passed 82, Failed 0, Skipped 0 y Errors 0, con una duración de 13 segundos y 327 milisegundos. Esta comprobación corresponde al proyecto ejecutado en su equipo.

Verificación	Resultado
Suite de integración	38 casos aprobados.
Colección con Newman	36 solicitudes; 82 comprobaciones aprobadas.
Colección en Postman del aprendiz	82 aprobadas; 0 fallidas; 0 errores.

Se verificaron acceso, validaciones, clientes, productos, entradas y salidas, cálculo de ventas, rechazo por stock insuficiente, anulación, bajas lógicas, reactivación y cierre de sesión. La suite automatizada también comprobó reversión completa de operaciones y escrituras simultáneas.

10. Evidencia de ejecución en Postman

Resumen de la colección JXSTech - GA7-AA5-EV03 ejecutada por el aprendiz. La imagen original se incluye en Capturas/Resumen_pruebas_EV03.jpeg.



Resultado visible: 82 comprobaciones aprobadas y 0 errores. Los códigos 401 y 409 de pruebas negativas representan rechazos esperados y se verifican como resultados correctos.

11. Versionamiento y archivos de entrega

El proyecto conserva el historial de EV01 y los cambios de EV03 en la rama aa5-ev03-jxstech. El enlace correspondiente al código se encuentra en ENLACE_REPOSITORIO.txt. El historial se consulta con git log.

La entrega debe contener el código, los archivos package.json y package-lock.json, la plantilla .env.example, la documentación y la colección de Postman. Las credenciales .env, las dependencias node_modules y el directorio interno .git quedan excluidos del comprimido.

12. Referencias técnicas

MongoDB. Transacciones con el driver de Node.js.

https://www.mongodb.com/docs/drivers/node/current/crud/transactions/

MongoDB. Requisitos de producción para transacciones.

https://www.mongodb.com/docs/manual/core/transactions-production-consideration/

MongoDB. API ClientSession del driver 7.8.

https://mongodb.github.io/node-mongodb-native/7.8/classes/ClientSession.html